

实验2 类定义和进一步使用

一. 实验目的

1. 掌握类的构造函数、拷贝构造函数
2. 掌握类对象的赋值和复制
3. 掌握对象的动态生成和释放
4. 能利用类和对象的知识，解决实际问题。

二. 实验内容

1. 阅读下面的程序与输出结果，添加一个拷贝构造函数来完善整个程序

```
#include<iostream>
using namespace std;
class Cat
{
public:
    Cat();
    ~Cat();
    int getage()const{return *itsage;}
    void setage(int age){*itsage=age;}
protected:
    int *itsage;
};
Cat::Cat()
{
    itsage=new int;
    *itsage=5;
}
Cat::~~Cat()
{
    delete itsage;
    itsage=NULL;
}
int main()
{
    Cat frisky;
    cout<<"frisky's age:"<<frisky.getage()<<endl;
    cout<<"setting frisky to 6...\n";
    frisky.setage(6);
    cout<<"creating boots from frisky\n";
```

```

    Cat boots(frisky);
    cout<<"frisky's age:"<<frisky.getage()<<endl;
    cout<<"boots'age:"<<boots.getage()<<endl;
    cout<<"setting frisky to 7...\n";
    frisky.setage(7);
    cout<<"frisky's age:"<<frisky.getage()<<endl;
    cout<<"boots'age:"<<boots.getage()<<endl;
return 0;
}

```

当添加了拷贝构造函数后，程序的运行结果为：

```

frisky's age:5
setting frisky to 6...
creating boots from frisky
frisky's age:6
boots'age:6
setting frisky to 7...
frisky's age:7
boots'age:6

```

2. 设计一个 **Account** 类，模拟银行账户。这个账户包含如下成员：

- 私有数据成员：姓名（**string name**）、账号（**int id**）、余额（**double balance**）
- 静态数据成员：账户总数 **count**、总余额 **totalBalance**
- 构造函数：带参的构造函数，初始化姓名、账号、余额，更新静态数据成员，输出“账户[姓名]创建成功”
- 析构函数：更新静态数据成员，输出“账户[姓名]已注销”
- 常成员函数：**getBalance()** 返回余额，**display()** 输出账户信息（姓名、账号、余额）
- 静态成员函数：**getCount()** 返回账户总数，**getTotalBalance()** 返回总余额
- 普通成员函数：存款 **deposit(double amt)**、取款 **withdraw(double amt)** 修改余额并同步更新总余额

在主函数中：

1) .创建一个包含 3 个账户的对象数组；2) .输出所有账户信息；3) .对某个账户进行存取款操作，并输出总余额。在主函数运行过程中，观察构造和析构的顺序。

3. 设计一个字符串类 **MyString**，用于管理动态字符串。该类包含以下成员：

- 私有数据成员：字符指针 **str**、长度 **len**
- 构造函数：字符串初始化，动态分配内存，提示已给出函数实现
- 析构函数：释放内存，并给出提示“析构[字符串内容]”
- 拷贝构造函数：实现深拷贝
- 赋值运算符重载：实现深拷贝
- 成员函数：**display()** 输出字符串，**getLen()** 返回长度
- 友元函数：**MyString concat(const MyString& s1, const MyString& s2)**，字符串连接，返

回两个字符串连接后的新对象（使用动态内存分配）

在主函数中，创建多个 `MyString` 对象，测试拷贝构造、赋值和连接功能，在此过程中，观察深拷贝的效果。

提示-构造函数的实现如下：

```
MyString (const char* s = "")
{ len = strlen(s);
  str = new char[len + 1]; //动态数组
  strcpy(str, s);
  cout << "构造: " << str << endl;
}
```

4. 设计一个简单的学生成绩管理系统，包含两个类 `Date` 和 `Student`，两个类的信息如下：

`Date` 类：私有数据成员：`year, month, day`

构造函数：初始化年、月、日

成员函数：`display()` 输出日期（格式：`YYYY-MM-DD`）

2. `Student` 类

私有数据成员：

`name (string)`：学生姓名

`id (int)`：学号

`scores (int*)`：动态分配的整数数组，存储 3 门课程的成绩（每个学生都有 3 门课）

静态数据成员：

`totalStudents (int)`：记录当前存在的学生总数

`totalScoreSum (int)`：记录所有学生所有课程的成绩总和（用于统计平均分）

成员函数：

构造函数：初始化姓名、学号，动态分配成绩数组并初始化为 0，更新静态成员

析构函数：释放成绩数组，更新静态成员

拷贝构造函数：深拷贝（复制姓名、学号、成绩数组），更新静态成员

赋值运算符重载：深拷贝，更新静态成员。注意：被赋值后的对象数据更新了，需重新计算总分 `totalScoreSum`

`setScore(int index, int score)`：设置某门课的成绩（ $0 \leq \text{index} < 3$ ，`score` 在 0~100 之间）

`display(const Date& d) const`：输出日期、学生信息（姓名、学号）及各科成绩

`getAverage() const`：返回该学生的平均分

静态成员函数：`getTotalStudents()` 学生总数、`getGlobalAverage()` 所有学生所有科目平均分
在主函数中：

1) 创建两个 `Student` 对象 `s1`（张三，1001）和 `s2`（李四，1002），并为它们设置成绩
显示 `s1` 和 `s2` 的信息

2) 使用拷贝构造函数创建 `s3` 作为 `s1` 的副本，然后修改 `s3` 的成绩（不影响 `s1`）

3) 使用赋值运算符将 `s2` 赋值给 `s4`，然后修改 `s4` 的成绩（不影响 `s2`）

4) 动态创建一个 `Student` 对象 `s5`，设置成绩后释放

- 5) 创建一个 `Student` 对象数组（至少 2 个元素），初始化并输出信息
- 6) 输出学生总数、所有学生成绩总和、全局平均分，验证静态成员的正确性

三. 实验要求

独立完成实验内容，按照实验报告要求提交。实验报告中应有本次实验的体会。